

Architecture documentation

Status: Conceptual documentation derived only from the participant-supplied project description. Component names and behavior have not been verified against source files. The diagrams intentionally omit undocumented services, brokers, databases, deployment platforms, or network protocols.

1. Centralized cloud architecture — Baseline A

All input data is transmitted to the cloud coordinator. The reported baseline uses the HEAVY LSTM Autoencoder and does not perform local edge inference or adaptive model selection.

flowchart LR

```
Input[Input data] -->|All data transmitted| Cloud[CloudCoordinator]
Cloud --> LSTM[LSTM Autoencoder - HEAVY]
LSTM --> Detection[Anomaly detection result]
Detection --> Metrics[MetricsCalculator / SystemMonitor]
```

2. Static edge architecture — Baseline B

The edge processor runs a fixed STANDARD Autoencoder locally. The reported configuration sends anomaly events rather than all input data to the cloud. The supplied description does not specify whether the cloud coordinator performs any further analysis of those events, so the diagram shows only event transmission and its monitoring role, not a further detection step.

flowchart LR

```
Input[Input data] --> Edge[EdgeProcessor]
Edge --> Autoencoder[Autoencoder - STANDARD fixed]
Autoencoder --> Decision[Anomaly result]
Decision -->|Anomaly events only| Cloud[CloudCoordinator]
Autoencoder --> Metrics[MetricsCalculator / SystemMonitor]
```

3. Proposed adaptive edge architecture

The supplied design describes an AdaptiveController applying a resource-aware utility decision mechanism and an EdgeProcessor selecting among LIGHT, STANDARD, and HEAVY models. It also describes decisions between local processing and transmission, and says the reported configuration sent anomaly events only. The specific utility equation, inputs, thresholds, policy order, and conditions for bypassing local inference are not provided and are not invented in the diagram.

flowchart LR

```
Input[Input data] --> Controller[AdaptiveController]
Resources[Available resource state] --> Controller
```

```

Controller -->|Resource-aware decision| Edge[EdgeProcessor]
Edge --> Light[IsolationForestModel - LIGHT]
Edge --> Standard[AutoencoderModel - STANDARD]
Edge --> Heavy[LSTMAutoencoderModel - HEAVY]
Light --> Result[Anomaly result]
Standard --> Result
Heavy --> Result
Result -->|Reported configuration: anomaly events only| Cloud[CloudCoordinator]
Result --> Metrics[MetricsCalculator / SystemMonitor]
Controller --> Metrics

```

The resource-state input is a conceptual representation of the resource awareness described by the participant. Which resource values are actually sampled and how they map to the utility decision must be verified from the controller implementation and logs.

4. Processing-flow view

flowchart TD

```

Start[Observation arrives] --> Mode{Configured architecture}
Mode -->|Centralized baseline| SendAll[Transmit all data to cloud]
SendAll --> CloudInference[CloudCoordinator uses HEAVY LSTM Autoencoder]
CloudInference --> Record[Record detection and system metrics]

Mode -->|Static edge baseline| FixedModel[EdgeProcessor uses fixed STANDARD Autoencoder]
FixedModel --> EventOnly[Transmit anomaly events only]
EventOnly --> Record

Mode -->|Adaptive edge approach| Assess[AdaptiveController applies resource-aware utility]
Assess --> Select[EdgeProcessor selects LIGHT, STANDARD, or HEAVY model]
Select --> Detect[Local anomaly detection / processing decision]
Detect --> AdaptiveEvents[Reported configuration transmits anomaly events only]
AdaptiveEvents --> Record

```

This flow does not imply that the adaptive controller is empirically verified to switch among all models. The supplied summary says the STANDARD model was primarily selected in the reported resource conditions.

5. Component responsibilities as described

Component	Described role	Not yet verified
AdaptiveController (src/adaptive/)	Resource-aware utility decisions for the adaptive approach.	Utility formula, weights, inputs, decision thresholds, cadence, fallbacks, and logged decisions.
EdgeProcessor (src/edge/)	Local processing and selection of a model for adaptive operation.	Model loading, feature handling, selected models in actual runs, event format, edge/cloud routing logic.
CloudCoordinator (src/cloud/)	Represents centralized cloud processing.	Exact interface, input format, cloud-side event handling, network timing instrumentation.
MetricsCalculator (src/evaluation/)	Calculates detection metrics.	Label mapping, metric formulas/averaging, threshold usage, AUC implementation, confusion matrix.
SystemMonitor (src/evaluation/)	Monitors system performance.	Hardware/process scope, sampling interval, aggregation, timing boundary, instrumentation overhead.

6. Model-selection logic

The project description assigns the following categories:

Model label	Described model	Intended profile
LIGHT	IsolationForestModel 1	Lightweight statistical anomaly detector.
STANDARD	AutoencoderModel	Autoencoder-based detector.
HEAVY	LSTMAutoencoderModel 1	LSTM autoencoder intended for sequence-based anomaly detection.

The adaptive policy is described as selecting a model or processing action according to a resource-aware utility mechanism. No actual equation or decision thresholds were provided. A publication-ready account must extract and report the exact utility components, their normalization and weights, model-specific expected cost and detection score, evaluation frequency, action semantics, and behavior when resources or inputs are missing. The relative LIGHT/STANDARD/HEAVY labels are descriptive design categories, not measured performance rankings unless confirmed in controlled benchmarks.

7. Data and communication flow

- **Centralized:** all input data is reportedly transmitted to the cloud for LSTM-autoencoder detection.
- **Static edge:** input is processed locally by the fixed standard autoencoder; anomaly events, not all input data, are reportedly transmitted.
- **Adaptive edge:** the controller and edge processor are described as adapting model/processing decisions; the reported configuration transmits anomaly events only.

The summary does not define the meaning of “data transmitted” (payload only versus protocol overhead, upstream only versus both directions, totals versus per observation), nor the exact event format. These are needed to interpret the reported bytes accurately.

8. Resource-awareness and evaluation considerations

A useful resource-aware evaluation should record the input, resource state, chosen action/model, detection score and thresholded result, inference time, end-to-end time, CPU and memory measurements, and bytes transmitted for each comparable evaluation unit. This is a recommendation for future reproducibility, not an assertion that these data are currently logged. The reported adaptive run selected STANDARD primarily and has identical reported detection measures to static edge; actual controller transitions and any benefit under alternative constraints remain unverified.

9. Known design and evidence limitations

1. The architecture is derived from a project summary, not inspected code.
2. Utility weights may require calibration for the target application, according to the supplied project notes.
3. The adaptive run reportedly uses STANDARD primarily; it may not exercise selection across the full model portfolio.
4. The centralized latency reportedly includes uplink, downlink, and inference, while edge latency is local inference. The reported figures therefore have mismatched boundaries and should not be presented as a controlled end-to-end comparison.
5. System performance and model quality cannot be assessed statistically without run counts, raw outputs, variability, and the evaluation dataset.
6. Synthetic data and reportedly fixed random seeds are project-summary statements pending verification against data-generation scripts, configuration, and experiment logs.